Artificial Intelligence & Machine Learning

A Comprehensive AI Personal Study Assistant & Engineering Handbook

Prepared By: Mariya Mathew
Department: AI & ML

Executive Overview & Curriculum Blueprint

This study guide and engineering handbook provides an in-depth, structured path for mastering Artificial Intelligence and Machine Learning. Designed from an AI Engineer's perspective, this personal study assistant covers mathematical foundations, classical machine learning algorithms, deep learning neural network architectures, generative AI and modern Large Language Models (LLMs), as well as MLOps deployment practices.

Roadmap & Module Breakdown

- Mathematical Foundations for AI & ML
- Supervised & Unsupervised Machine Learning
- Deep Learning Mechanics & Neural Network Architectures
- Natural Language Processing & Transformers
- Generative AI, RAG & Large Language Models
- Reinforcement Learning Principles
- MLOps, Model Optimization & Production Deployment

Module 1: Mathematical Foundations for AI & ML

1.1 Linear Algebra

Linear algebra provides the mathematical framework for representing data and operations as vectors and matrices in high-dimensional spaces.

- **Vectors & Matrices:** Data points are represented as vectors $x \in \mathbb{R}^d$, while datasets and weight transformations are represented as matrices $A \in \mathbb{R}^{m \times n}$.
- **Matrix Multiplication:** Transformation $y = Ax + b$ maps inputs to feature spaces.
- **Eigenvalues & Eigenvectors:** Solves $Av = \lambda v$, providing principal direction dynamics critical for dimensionality reduction techniques like Principal Component Analysis (PCA).
- **Singular Value Decomposition (SVD):** Decomposes matrix $A = U \Sigma V^T$, essential for latent semantic analysis and low-rank matrix approximations.

```
import numpy as np

# Eigenvalue decomposition in NumPy

A = np.array([[4, 2], [1, 3]])

eigenvalues, eigenvectors = np.linalg.eig(A)

    print("Eigenvalues:", eigenvalues)

    print("Eigenvectors:\n", eigenvectors)
```

1.2 Multivariate Calculus & Optimization

Calculus enables network optimization by computing gradients across loss surfaces.

- **Partial Derivatives & Gradients:** The gradient $\nabla f(x)$ points in the direction of steepest ascent.
- **Jacobians & Hessians:** First-order and second-order partial derivative matrices used for complex surface optimization.
- **Gradient Descent:** Iterative optimization parameter updates:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \nabla_\theta L(\theta)$$

1.3 Probability & Statistics

- **Bayes' Theorem:** Basis for probabilistic modeling:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$
|---|Bernoulli, Categorical, Poisson, and Beta distributions form the assumptions behind maximum likelihood estimation (MLE).

Module 2: Supervised & Unsupervised Machine Learning

2.1 Supervised Learning Framework

Supervised algorithms learn a mapping function $f: X \to Y$ given labeled training pairs (x_i, y_i).

Algorithm Family	Primary Use Case	Objective / Loss Function	Strengths
Linear / Logistic Regression	Continuous/Binary Prediction	Mean Squared Error (MSE) / Binary Cross-Entropy	Interpretable, fast inference
Support Vector Machines (SVM)	Classification / Regression	Hinge Loss + L_2 Regularization	Effective in high dimensions
Decision Trees & Random Forests	Non-linear tabular data	Gini Impurity / Information Gain	Handles mixed data, feature importance
Gradient Boosted Trees (XGBoost/LightGBM)	High-accuracy tabular modeling	Gradient-based loss minimization	SOTA accuracy on structured data

```
from sklearn.ensemble import GradientBoostingClassifier

from sklearn.metrics import accuracy_score, precision_recall_fscore_support

# Model initialization & fitting pipeline

clf = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=5, random_state=42)

clf.fit(X_train, y_train)
```

```
y_pred = clf.predict(X_test)

acc = accuracy_score(y_test, y_pred) print(f"Test

Accuracy: {acc:.4f}")
```

2.2 Unsupervised Learning & Clustering

- **K-Means Clustering:** Iterative centroid assignment minimizing intra-cluster variance (Inertia).
- **Dimensionality Reduction (PCA):** Projects data onto orthogonal axes that maximize variance.

Module 3: Deep Learning Mechanics & Neural Network Architectures

3.1 Artificial Neural Networks (ANNs) & Backpropagation

Deep learning constructs complex functional approximations using stacked layers of computational nodes.n

```
import torch
import torch.nn as nn import
torch.optim as optim

class DeepMLP(nn.Module):
def init(self, input_dim: int, hidden_dim: int, num_classes: int): super(DeepMLP,
self).init()
self.layer_stack = nn.Sequential(
nn.Linear(input_dim, hidden_dim),
nn.BatchNorm1d(hidden_dim),
nn.GELU(),
nn.Dropout(0.2),
nn.Linear(hidden_dim, hidden_dim),
nn.BatchNorm1d(hidden_dim),
nn.GELU(),
nn.Linear(hidden_dim,  num_classes)
)def forward(self, x: torch.Tensor) -> torch.Tensor:

    return self.laye
r_stack(x)
```

3.2 Convolutional Neural Networks (CNNs) & Computer Vision

CNNs preserve spatial structures through weight sharing and kernel convolutions.

* **Convolution Layer:** Extracts localized spatial features (edges, textures, shapes).

* **Pooling Layers:** Max/Average pooling reduces spatial dimension and offers translation invariance.

* **Architectures:** ResNet (Residual connections solving vanishing gradient), EfficientNet, and Vision Transformers (ViT).

3.3 Recurrent Architectures & Sequence Models

* **LSTMs & GRUs:** Gated memory cells that maintain long-term context while controlling information flow via input, forget, and output gates.

Module 4: Natural Language Processing & Transformers

4.1 Transformer Architecture & Self-Attention

Transformers replaced sequential recurrence with parallelized self-attention mechanisms.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where:

* Q: Query matrix ($N \times d_k$)

* K: Key matrix ($M \times d_k$)

* V: Value matrix ($M \times d_v$)

* $\sqrt{d_k}$: Scaling factor preventing vanishing gradients in softmax.

```python

import torch

import torch.nn.functional as F
```

```
def scaled_dot_product_attention(Q, K, V, mask=None): d_k =
    Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / torch.sqrt(torch.tensor(d_k, dtype=torch.float32))

    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)

    attention_weights = F.softmax(scores, dim=-1)
    output = torch.matmul(attention_weights, V)
    return output, attention_weights
```

Module 5: Generative AI, RAG & Large Language Models

5.1 Generative AI Architectures

- **Variational Autoencoders (VAEs):** Encode inputs into probabilistic latent distributions $q_\phi(z|x)$.
- **Diffusion Models:** Noise addition and iterative denoising via learned score functions.
- **Autoregressive Large Language Models (LLMs):** Predict the next token given preceding context $P(w_t | w_1, w_2, \dots, w_{t-1})$.

5.2 Retrieval-Augmented Generation (RAG) Architecture

RAG integrates parametric knowledge (LLM parameters) with non-parametric external knowledge stores (Vector Databases).[User Query] ---> [Embedding Model] ---> [Vector Database Search]

|

v

[LLM Response] <--- [Context Prompt] <--- [Retrieved Top-K Chunks]

Core Components for RAG Systems:

1. **Document Ingestion:** Chunking raw documents into optimal context windows (e.g., 512 tokens with 50-token overlap).
2. **Dense Vector Embeddings:** Mapping chunks to high-dimensional embedding space using models like OpenAI `text-embedding-3-small` or HuggingFace `bge-large-en`.
3. **Similarity Search:** Query execution via HNSW or Cosine Distance indexing.

Module 6: Reinforcement Learning Principles

6.1 Markov Decision Processes (MDP)

Reinforcement learning optimizes an agent's sequential actions within an environment to maximize cumulative reward $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.

- **State (S):** Current environment representation.
- **Action (A):** Decision executed by agent.
- **Reward (R):** Feedback scalar from environment.
- **Policy ($\pi(a|s)$):** Mapping from state to action probabilities.

6.2 Deep RL Frameworks

- **Deep Q-Networks (DQN):** Q-value approximation using neural networks with experience replay.
- **Proximal Policy Optimization (PPO):** Policy gradient algorithm restricting step sizes to improve stability, foundational for RLHF (Reinforcement Learning from Human Feedback) in alignment of LLMs.

Module 7: MLOps, Model Optimization & Production Deployment

7.1 Model Optimization Techniques

- **Quantization:** Converting floating-point weights (FP32) to low-bit precision (INT8/INT4) to reduce memory bandwidth demands.
- **Pruning:** Removing redundant zero-weight parameters to reduce computation graph size.
- **Knowledge Distillation:** Training a smaller student network using soft labels generated by a larger teacher model.

7.2 Production Serving Infrastructure

- **Triton Inference Server / vLLM:** Optimized high-throughput LLM serving using Continuous Batching and PagedAttention.
- **Model Monitoring:** Tracking data drift, concept drift, and response latency across production microservices.

End of Personal Study Assistant - Mariya Mathew, Department of AI & ML